AI SalesGrid
Your AI Superpower in Sales for Enterprise

On AWS

The Problem: B2B sales take weeks of back-and-forth emails and multi asking questions to align on pricing, rules, and proposals.

In large companies:
- Sales promise features that don't exist
- Offers violate company pricing rules
- Proposals don't follow legal/financial constraints
- Managers don't have real-time visibility
- CEO gets incomplete reports
- Everything is manual, slow, and inconsistent

This causes:
- Revenue leakage
- Legal risk
- Pricing mistakes
- Lost deals
- Internal conflict

The Solution: A multi-agent system where a client or sales rep talks to a **Amazon Nova and AWS Bedrock** that orchestrates 5+ background agents to validate, price, and report in seconds.

Real-Time AI Multi-Agent Sales Orchestrator
Your AI Superpower in Sales – Real-Time Enterprise Deal Engine

A voice-enabled, multi-agent system where:
- Sales talks naturally (can interrupt)
- AI validates offers instantly
- AI builds proposals
- AI checks rules automatically
- AI generates executive reports
- Everything is real-time

Powered by:
- Amazon Nova (voice + interruption)
- AWS Bedrock multi-agent orchestration
- AWS Bedrock Flow
- AWS hosting
- Knowledge Base (Bedrock Knowledge base/ OpenSearch)
- AWS API Gateway
- AWS Lambda
- AWS ECS

End-to-End Workflow

The system operates in the following sequence:

Step 1

User speaks through the web application.

Step 2

Audio is streamed to **AWS API Gateway Connect with AWS Bedrock**.

Step 3

Gemini processes speech and forwards the request.

Step 4

The request reaches the **Account Manager Agent**.

Step 5

The Head Agent orchestrates the sub agents:

- Solution Architect Agent
- Pricing Agent
- Risk Agent
- Proposal Agent
- Executive Report Agent

Step 6

Agents retrieve relevant knowledge using:

AWS Bedrock.

Step 7

Deal data is retrieved from:

Dynamodb / RDS / CRM.

Step 8

Results are consolidated by the **Summarizer Agent**.

Step 9

Outputs are generated:

- Proposal document
- Executive report
- CRM updates
- Real-time response to the user.

Security Considerations

Security measures include:

- IAM role-based access control
- Data encryption at rest and in transit
- Secure API authentication
- Audit logging via Cloud Trail

Scalability

The architecture is designed for high scalability.

Benefits include:

- Serverless deployment via Lambda and ECS
- Automatic scaling based on traffic
- Multi-agent parallel processing
- Real-time streaming responses

Multi-Agent Flow

User (voice)
↓
Account Manager Agent
↓
Solution Architect Agent
↓
Pricing Agent
↓
Risk Agent
↓
Proposal Agent
↓
Executive Report Agent
↓
Summarizer Agent

System Architecture (AWS)

Brain Layer

- AWS API Gateway
- AWS Bedrock for multi-agent routing with Amazon Nova

Knowledge Base

- S3 (rules)
- RDS (pricing models)
- S3 (documents)

Backend

- Lambda (microservices per agent)
- AWS API Gateway (agent communication)
- Lambda (trigger workflows)

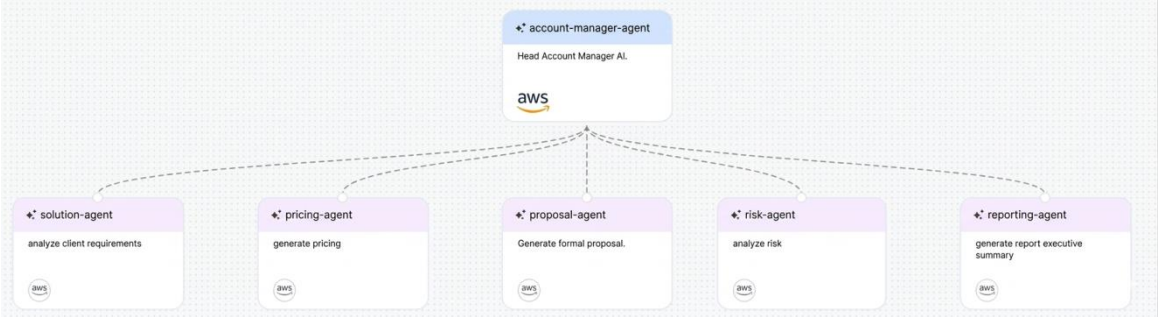
Reporting

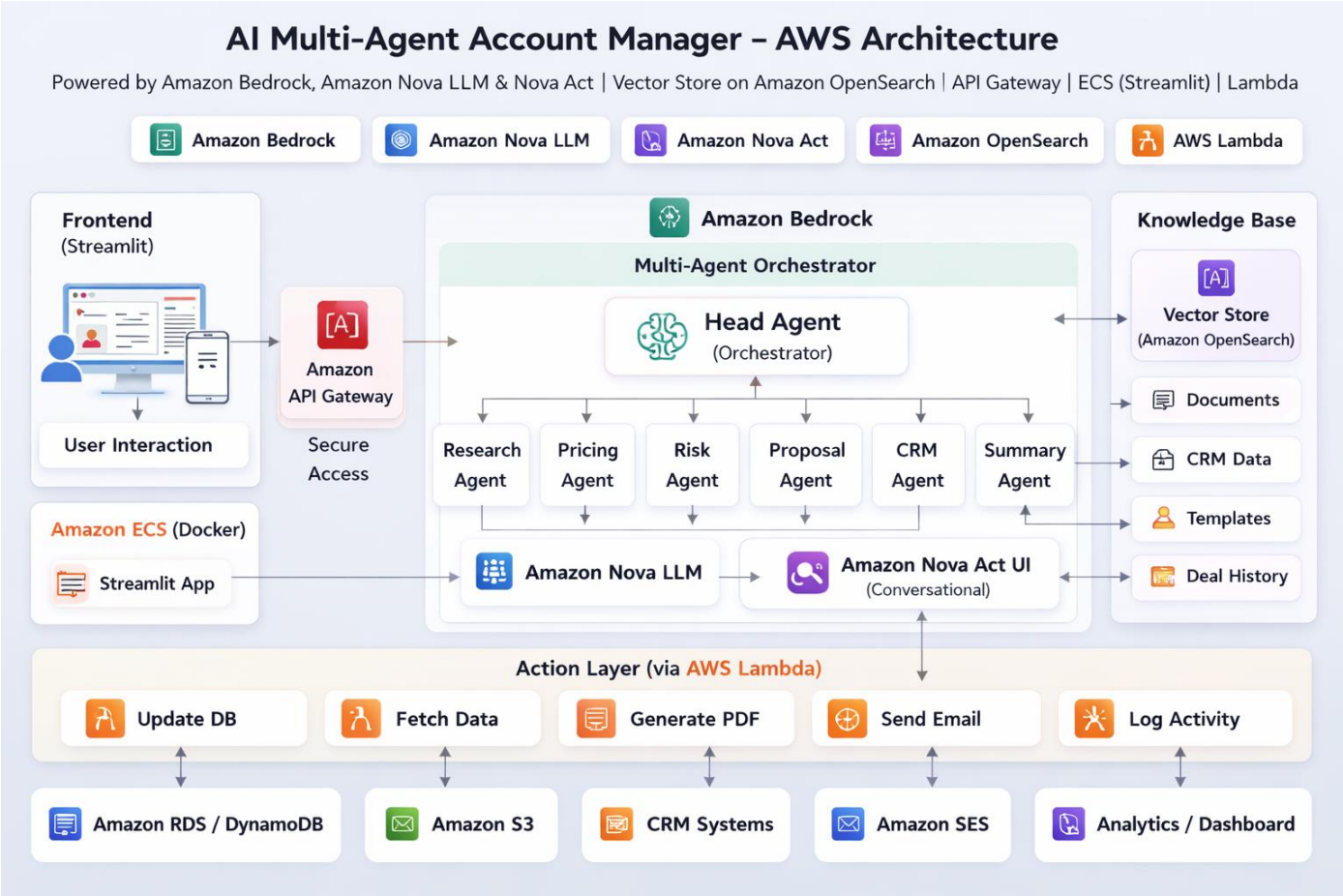
- Redshift
- Dynamodb

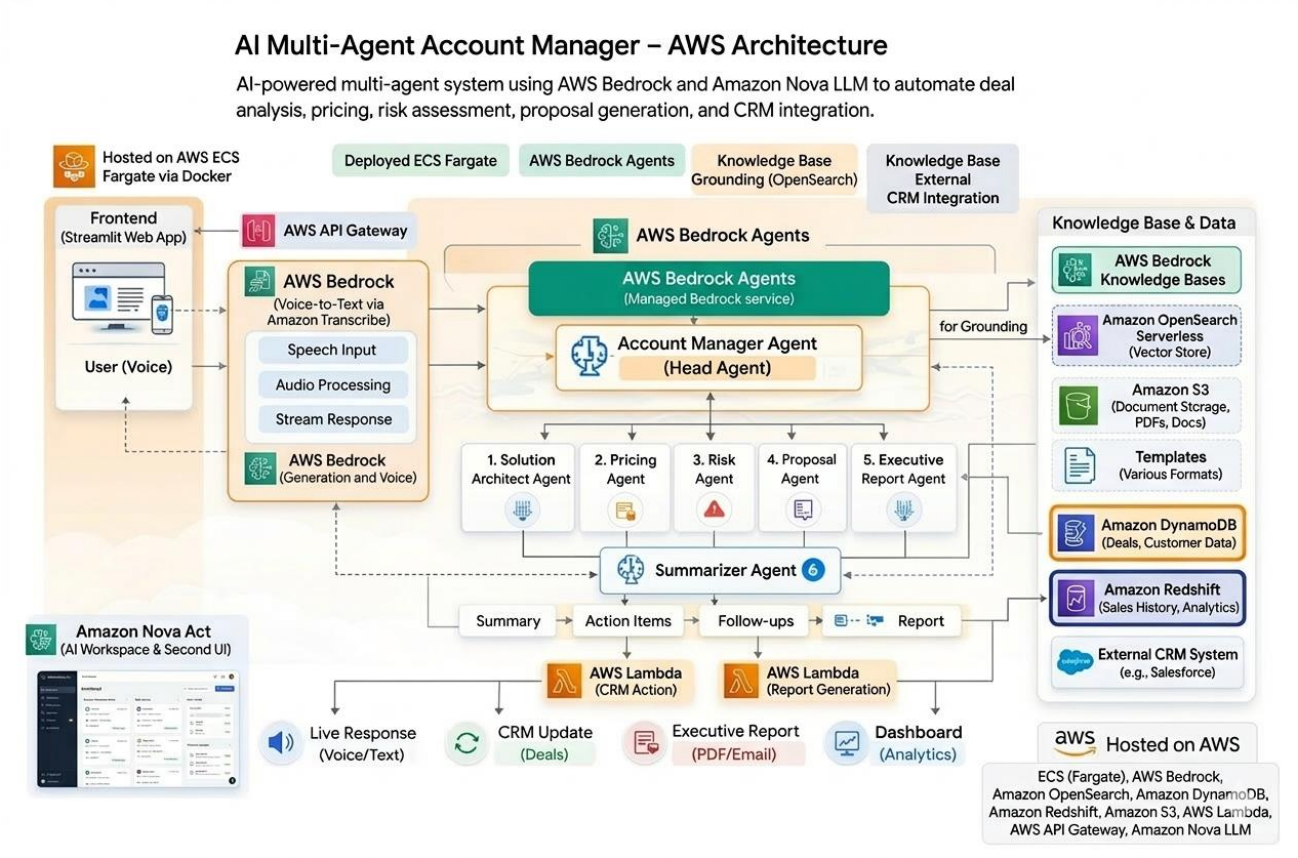
FINAL ARCHITECTURE OVERVIEW

Frontend (Web App)
↓ (WebSocket Audio Stream)
Streamlit or Amazon Nova Act
↓
Account Manager Agent (AWS API Gateway + AWS Bedrock + Amazon Nova)
↓
Sub Agents (Pricing / Proposal / Risk / Reporting)
↓
AWS Bedrock (Knowledge Base grounding)
↓
RDS / Dynamodb / Redshift / OpenSearch (Deal storage)

ARCHITECTURE







This architecture component diagram details a fully managed solution on AWS

AWS Multi-Agent AI Architecture

This architecture component diagram illustrates a fully managed AI solution on **AWS** that integrates a **multi-agent orchestration framework powered by foundation models** to automate account management tasks such as deal analysis, pricing evaluation, proposal generation, and CRM integration.

Key features of this system include:

• AI-Driven Interaction Layer

The system enables conversational and text-based interaction through a web interface built with Streamlit and deployed in containers on **Amazon ECS**.
User requests are securely routed through **Amazon API Gateway**, which exposes REST or WebSocket endpoints that connect the frontend with backend AI agents.

• Multi-Agent Orchestration

A **Head Agent** orchestrates multiple specialized AI agents using **Amazon Bedrock** with **Amazon Nova LLM models**.
The head agent coordinates tasks and delegates them to domain-specific agents including:

- Solution Architect Agent
- Pricing Agent
- Risk Assessment Agent
- Proposal Generation Agent
- CRM Agent
- Executive Report Agent

Each agent independently processes its task while contributing results to the overall workflow.

• Knowledge Base Grounding (RAG)

Agents retrieve contextual enterprise knowledge using **retrieval-augmented generation (RAG)**.
Documents such as company policies, pricing rules, templates, and CRM data are embedded and stored in a **vector database on Amazon OpenSearch Service**.

This enables agents to query relevant information dynamically and generate accurate responses grounded in company knowledge.

• Action & System Integration Layer

Agents trigger operational workflows through **AWS Lambda**, enabling the system to interact with backend services such as:

- Updating CRM or deal databases
- Fetching customer or sales history
- Generating reports or proposals
- Sending notifications or emails
- Logging activity and analytics

Lambda functions provide a scalable serverless execution layer that connects AI reasoning with real business actions.

• Multi-Channel AI Interface

In addition to the Streamlit frontend, the architecture integrates **Amazon Nova Act** as a conversational interface for advanced AI interactions and automation workflows.

• Post-Processing & Reporting

A **Summarizer Agent** consolidates the outputs from all specialized agents into structured outputs including:

- Deal summaries
- Recommended actions
- Follow-up tasks
- Executive reports

These outputs can be delivered through dashboards, exported reports, or CRM updates.

Technical Implementation on AWS

• Orchestration

Use **multi-agent orchestration logic built on Amazon Bedrock** with **Amazon Nova foundation models** to coordinate agent collaboration and reasoning workflows.

• Knowledge Base

Store enterprise documents in **Amazon S3**, generate embeddings, and index them in **Amazon OpenSearch Service** for vector search and RAG retrieval.

• API Layer

Expose backend AI capabilities through **Amazon API Gateway**, enabling secure communication between the frontend application and the AI agents.

• Actions & Automation

Use **AWS Lambda** to execute operational tasks triggered by agents, including database updates, integrations with CRM systems, and report generation.

• Hosting

Deploy the frontend **Streamlit application** in Docker containers on **Amazon ECS** for scalable containerized hosting.

Step-by-Step Execution Summary

1. Data Preparation

- Upload company knowledge documents (PDFs, policies, pricing rules) to **Amazon S3**.

2. Knowledge Base Setup

- Generate embeddings and index documents into **Amazon OpenSearch Service vector store**.

3. Agent Framework

- Build the **multi-agent orchestration system using Amazon Bedrock and Amazon Nova models**.
- Implement the Head Agent and specialized task agents.

4. Backend API

- Create API endpoints using **Amazon API Gateway** to allow the frontend to communicate with AI agents.

5. Action Functions

- Implement business logic using **AWS Lambda functions** to perform system actions (DB updates, CRM integration, report generation).

6. Frontend Deployment

- Containerize the Streamlit UI and deploy it on **Amazon ECS**.

7. AI Interaction

Integrate **Amazon Nova Act** as a conversational interface for advanced AI interactions and automation workflows.

AI Multi-Agent Account Manager — AWS Implementation Guide

STEP 1 — Create AWS Environment

1. Sign in to the **AWS Management Console**
2. Create a dedicated environment for the solution

Example naming:

Project Name: live-sales-ai
Environment: production / staging

STEP 2 — Enable Required AWS Services

Ensure the following services are available in your AWS region.

Core AI & Application Services:

• **Amazon Bedrock**
• **Amazon Nova LLM**
• **Amazon Nova Act**
• **Amazon OpenSearch Service**

Application & Integration Services:

• **Amazon API Gateway**
• **AWS Lambda**
• **Amazon ECS (Fargate)**
• **Amazon S3**

Data & Analytics:

• **Amazon DynamoDB**
• **Amazon RDS** (optional relational storage)
• **Amazon Athena / QuickSight** for analytics

Security & Access:

• **AWS IAM**
• **Amazon CloudWatch**

STEP 3 — Configure Amazon Bedrock (AI Platform)

Navigate to:

AWS Console → Amazon Bedrock

Enable model access for:

- **Amazon Nova LLM**
- **Amazon Nova Embeddings**

These models will power:

- Multi-Agent reasoning
- Retrieval-Augmented Generation (RAG)
- Proposal generation
- Risk evaluation

STEP 4 — Create Knowledge Base (Company Rules)

The knowledge base stores company policy and sales rules.

Example documents:

- Pricing rules
- Discount policies
- SLA agreements
- Legal constraints
- Service catalog

4.1 Create Document Storage

Navigate to:

Amazon S3 → Create Bucket

Bucket name:

company-rules-kb

Upload files

4.2 Create Vector Store (Semantic Search)

Navigate to:

Amazon OpenSearch Service → Create Domain

Enable:

Vector Search

Data Flow:

S3 documents
↓
Embedding generation

↓
OpenSearch vector index

This enables:

• semantic document retrieval
• knowledge grounding
• contextual reasoning for agents

STEP 5 — Create Multi-Agent System (Amazon Bedrock)

Amazon Bedrock will orchestrate the agents.

Architecture:

Head Agent (Account Manager)
↓
Sub-Agents

Agents communicate using:

Bedrock orchestration + tool calling

Agent 1 — Account Manager (Head Agent)

Configuration:

Name: account-manager-agent
Model: Amazon Nova LLM

Responsibilities:

• understand client request
• delegate tasks to agents
• aggregate responses
• generate final summary

Agent 2 — Solution Architect Agent

Purpose:

Designs technical solutions based on customer requirements.

Agent 3 — Pricing Agent

Purpose:

Validates pricing and discount policies.

Agent 4 — Proposal Generator

Purpose:

Generates formal proposals.

Outputs:
PDF proposal Stored in: Amazon S3

Agent 5 — Risk Agent

Purpose:

Evaluates deal risk.

Outputs:

risk score and approval recommendation

Agent 6 — Reporting Agent

Purpose:

Generates executive reports.

Data sources:

DynamoDB
OpenSearch
S3

STEP 6 — Connect Agents Together

Agent orchestration flow:

User Request
↓
Account Manager Agent
↓
Delegates to:
- Solution Agent
- Pricing Agent
- Risk Agent
- Proposal Agent

↓
Summarizer Agent

↓
Final response

STEP 7 — API Layer (Amazon API Gateway)

The API layer connects the **frontend** to the AI agents.

Navigate to:

Amazon API Gateway → Create API

Flow:
Frontend
↓
API Gateway
↓
Lambda / Bedrock
↓
Agents

STEP 8 — Action Layer (AWS Lambda)

Lambda executes actions triggered by agents.

Function	Purpose
updateDeal	store deal
generateProposal	create proposal
sendEmail	send proposal
logActivity	audit

Lambda tasks:

- update CRM
- generate PDF proposal
- store deal data
- trigger emails

STEP 9 — Database for Deals

Create database for deal tracking.

Option 1:

Amazon DynamoDB

Option 2:

Amazon RDS (PostgreSQL) Used for structured reporting.

STEP 10 — Frontend (Streamlit Application)

The frontend provides a conversational interface.

Deployment architecture:

Docker container
↓
Amazon ECS (Fargate)

Streamlit features:

- chat interface
- microphone input
- real-time transcript
- proposal viewer
- deal dashboard

Flow:
User
↓
Streamlit UI
↓
API Gateway
↓
Bedrock Agents

STEP 11 — Secondary AI Interface (Amazon Nova Act)

Amazon Nova Act provides a conversational AI UI.

Capabilities:

- AI assistant interface
- executive interaction
- natural language queries

STEP 12 — Analytics Dashboard

Sales analytics pipeline:

DynamoDB / RDS
↓
Amazon Athena
↓
Amazon QuickSight

Final AWS Architecture Flow

User
↓
Streamlit Frontend (ECS)
↓
API Gateway
↓
Amazon Bedrock (Nova LLM)
↓
Multi-Agent Orchestrator
↓
Sub Agents
- Solution
- Pricing
- Risk
- Proposal
- Reporting

↓
Knowledge Base
- S3 documents
- OpenSearch vector store

↓
Lambda Action Layer
↓
Databases
- DynamoDB
- RDS

↓
Analytics
- Athena
- QuickSight

Hosting all Infrastructure on AWS

AI SalesGrid
Your AI Superpower in Sales for Enterprise

On AWS

Developed by: Fady Nabil
https://github.com/fadynabil10